SYSTEM DESIGN & FEATURE WORKFLOW

# درب

## التصميم المعماري ومنهج بناء الميزات

دليل عملي لمشروع دورة «تمكين»: من فهم المعمارية، إلى بناء أي ميزة بخطوات ثابتة، إلى تسليمها مدموجة ومختبرة.

| | |
|---|---|
| المطوّر | Rashed eybo |
| المستودع | github.com/Rashedx1m/dareb-flutter |
| الحالة عند الكتابة | الفرع main عند e5002b8، و159 اختباراً ناجحاً |
| الإصدار | 1.0، أكتوبر 2026 |

## كيف تقرأ هذا الدليل

الدليل في خمسة أجزاء. **الجزء الأول** يشرح المعمارية كما هي في مشروعك: لماذا قُسّم الكود هكذا، وكيف يمر الطلب من الزر إلى الخادم ويعود. **الجزء الثاني** هو المنهج نفسه: ست خطوات ثابتة تبني بها أي ميزة، وفي كل خطوة الكود العام لميزة CRUD كاملة. **الجزء الثالث** يضيف حالات الاستخدام الأخرى: البحث، والصفحات، ورفع الصور، والتخزين المحلي. **الجزء الرابع** مرجع سريع بالقواعد والأسماء وقائمة الانتهاء. **الجزء الخامس** يلخص ما بنيناه في هذه الدورة، وأين وصلت، وما الخطوة التالية.

### عن الكود في هذا الدليل

كل كود الجزأين الثاني والثالث كُتب أولاً ميزةً حقيقية باسم `items` داخل المشروع، ثم مرّ على `flutter analyze` بلا أي ملاحظة، وعلى 16 اختباراً خاصاً به، وعلى كامل اختبارات المشروع (175 اختباراً). بعدها حُذفت الميزة ليبقى المشروع كما هو. أي أن الكود هنا يعمل كما هو، لا مجرد مثال نظري.

عند الاستخدام: استبدل `Item` و `items` باسم ميزتك ( `Comment` ، `Album` ...)، وأسماء الحقول والمسارات بما رأيته في رد الخادم الحقيقي في الخطوة 0.

# المحتويات


**الجزء الأول: الصورة الكبيرة**

1. المشروع في سطور
2. المعمارية النظيفة
3. هيكل المجلدات
4. أساس core
5. رحلة الخطأ

**الجزء الثاني: المنهج: ست خطوات لكل ميزة**

الخطوة 0: الفهم قبل الكود

الخطوة 1: Domain

الخطوة 2: Data

الخطوة 3: Cubit وState

الخطوة 4: DI والمسار والواجهة

الخطوة 5: التحقق والتسليم

**الجزء الثالث: حالات استخدام إضافية**

البحث أثناء الكتابة

القوائم الطويلة: صفحةً صفحة

رفع صورة

العمل بلا إنترنت: تخزين آخر قائمة

تحويل ميزة تجريبية إلى الخادم

**الجزء الرابع: المرجع السريع**

قواعد الاعتماد

الأسماء

قائمة الانتهاء

أخطاء صادفناها، ودرس كل منها

**الجزء الخامس: رحلتنا في هذه الدورة، وما بعدها**

ما بنيناه في هذه الجلسة

أين وصلت

الطريق إلى تسليم المشروع

قبل التسليم

مسرد المصطلحات

الجزء الأول

# الصورة الكبيرة

قبل أن تكتب سطراً: ما المشروع، وكيف قُسّم، ولماذا.

## 1. المشروع في سطور

«درب» تطبيق تعليمي بـ Flutter فيه المواد والدورات والاختبارات والاشتراكات. الواجهة الخلفية Laravel على خادم التدريب، والتطبيق مبني بـ **Clean Architecture** لكل ميزة على حدة. التطبيق عربي من اليمين إلى اليسار، بخط Noto Sans Arabic للنص وLato للأرقام.

| الحزمة | دورها في المشروع |
|---|---|
| `flutter_bloc` | إدارة الحالة بـ Cubit (وBloc في الدورات والاختبارات) |
| `get_it` | حقن الاعتماديات: من ينشئ ماذا، في `core/di/injection.dart` |
| `dartz` | `Either<Failure, T>` : كل نتيجة إما فشل برسالته أو قيمة |
| `equatable` | مساواة بالقيم للكيانات والحالات، فلا يُعاد بناء الواجهة بلا تغيير |
| `go_router` | المسارات، وحارس يمنع الصفحات الداخلية بلا جلسة |
| `dio` | الاتصال بالخادم، ملفوفاً كله في `ApiClient` |
| `shared_preferences` | حفظ الجلسة (التوكن والمستخدم) على الجهاز |
| `image_picker` ، `url_launcher` | اختيار الصور ورفعها، وفتح الروابط الخارجية |

حالة الميزات عند كتابة هذا الدليل:

| الميزة | المصدر | طلبات الخادم |
|---|---|---|
| الدخول والتسجيل | الخادم | `auth/login` ، `auth/active` ، `auth/logout` ، `users/edit` ، المحافظات والمدارس والفروع |
| الملف الشخصي | الخادم | `profile/get` ، `users/update` ، `users/update-image` |
| الدعم | الخادم | `contact-us/add` ، `infos/all` |
| الاشتراكات | الخادم | `packages/all` ، `packages/details` ، `payment-methods/all` ، `subscriptions/add` |
| الإشعارات | الخادم | `notifications/get` |

| الميزة | المصدر | طلبات الخادم |
|---|---|---|
| الدورات، الاختبارات، أخبار الرئيسية، المعرض، الإنجازات | تجريبي | بيانات من `MockCatalog` عبر `*MockDataSource` |
| التنزيلات | تجريبي | ملفات على الجهاز، لا تحتاج الخادم |

## 2. المعمارية النظيفة

كل ميزة مقسومة إلى ثلاث طبقات، ولكل طبقة سؤال واحد تجيب عنه. القاعدة الذهبية: **الاعتماد يتجه إلى الداخل فقط**، أي أن الطبقتين الخارجيتين تعرفان الـ domain، والـ domain لا يعرف أحداً.




| الطبقة | تعرف | لا تعرف | لماذا |
|---|---|---|---|
| domain | Dart فقط، و `core` (الأخطاء والـ UseCase والتحقق) | Flutter، JSON، الخادم، الواجهة | أثبت جزء: قواعد الميزة لا تتغير إن تغيّر الخادم أو التصميم |
| data | domain و `ApiClient` | الواجهة | إن غيّر الخادم اسم حقل تعدّل `fromJson` وحده |
| presentation | domain (الـ UseCases والكيانات) | data: لا Model ولا DataSource ولا Repository | تُختبر بمستودع مزيّف بلا خادم |

> **قاعدة واحدة تكفي لتتذكر الباقي**
>
> اسأل عند كل `import` : هل أستورد من طبقة «أخرج» مني؟ إن كان نعم فهو خطأ. الصفحة لا تستورد Repository، والـ Entity لا يستورد Flutter، والـ Model لا يعرف الـ Cubit.

### رحلة الطلب والرد

خذ زر «إضافة» مثالاً. الطلب ينزل من الأعلى إلى الخادم، والرد يصعد بالعكس متحولاً في كل طبقة:




## 3. هيكل المجلدات

هيكل المشروع

```
lib/
  main.dart · app.dart
  core/
    constants/app_strings.dart      كل نصوص الواجهة
    theme/                          app_colors · app_icons · app_theme · app_assets
    widgets/                        PageHeader · ErrorView · LoadingView · app_sheet …
    network/                        api_client · api_config · session_expiry
    error/                          exceptions · failures · guard
    usecase/usecase.dart            UseCase<T, Params> و NoParams
    utils/                          validators · external_link
    di/injection.dart               تسجيل كل ميزة في دالة _initX
    router/app_router.dart          المسارات والحارس
  features/<feature>/
    domain/     entities/ · repositories/ · usecases/     ملف لكل UseCase
    data/       models/ · datasources/ · repositories/    ملف لكل Model
    presentation/  cubit/ (x_cubit + x_state) · pages/ · widgets/
test/
  helpers/fake_http_adapter.dart    خادم مزيّف للاختبارات
  features/<feature>/  <feature>_data_test · <feature>_cubits_test · <feature>_flow_test
```

## 4. أساس core

هذه الأجزاء مشتركة بين كل الميزات. لن تكتبها من جديد، لكن عليك أن تفهم ما تفعله لأن كل ميزة تمر عبرها.

### ApiClient: البوابة الوحيدة إلى الخادم

كل طلب في التطبيق يمر من `ApiClient` وحده. يضيف ترويسة `Accept: application/json` وتوكن الجلسة، ويفك غلاف الرد `{message, data}` فيعيد `data` مباشرة، ويحوّل كل خطأ إلى استثناء برسالة عربية. له ثلاث دوال: `get` و`post` (الحقول form-data كما في Postman) و`upload` لرفع ملف. مرّر `parse:` في كل طلب يعيد بيانات: إن جاء الرد بشكل غير متوقع يصير «رد غير متوقع من الخادم» بدل أن تتجمد الشاشة.

| ما حدث | الاستثناء | الرسالة التي تصل المستخدم |
|---|---|---|
| لا رد (لا إنترنت، مهلة) | `ServerException` | تعذّر الاتصال بالخادم، تحقق من اتصالك بالإنترنت |
| 401 على طلب بتوكن | `AuthException` | انتهت الجلسة، سجّل الدخول من جديد (ويُمسح الدخول وتفتح صفحة الدخول) |
| 422 أخطاء حقول | `ValidationException` | أول رسالة في `errors` كما كتبها الخادم |
| 404 | `ServerException` | المحتوى المطلوب غير موجود |
| 400 أو غيره | `ServerException` | `message` الخادم كما هو |
| شكل الرد لا يطابق الـ Model | `ServerException` | رد غير متوقع من الخادم |

## الأخطاء: Exception في data، وFailure في domain

طبقة data ترمي **استثناءات**. المستودع يحوّلها إلى **Failure** عبر `guard` ، فلا يرى الـ Cubit استثناءً أبداً، بل `Either<Failure, T>` فقط. لهذا لا يوجد `try/catch` في أي مستودع أو Cubit.

lib/core/error/guard.dart

```dart
/// ينفّذ استدعاء مصدر البيانات ويحوّل استثناءاته إلى [Failure]، حتى لا يتكرر
/// `try/catch` في كل دالة من دوال المستودعات.
Future<Either<Failure, T>> guard<T>(Future<T> Function() action) async {
  try {
    return Right(await action());
  } on ServerException catch (e) {
    return Left(ServerFailure(e.message));
  } on CacheException catch (e) {
    return Left(CacheFailure(e.message));
  } on AuthException catch (e) {
    return Left(AuthFailure(e.message));
  } on ValidationException catch (e) {
    return Left(ValidationFailure(e.message));
  } on Exception {
    // استثناء من خارج طبقة البيانات (التخزين المحلي مثلاً). بدونه لا يصل الـ Cubit
    // ظاهرة (Error) تبقى. الأخطاء البرمجية. فتبقى الشاشة على التحميل رد
    return const Left(ServerFailure('حدث خطأ غير متوقع'));
  }
}
```

## UseCase: فعل واحد بمعامل واحد

lib/core/usecase/usecase.dart

```dart
abstract class UseCase<T, Params> {
  Future<Either<Failure, T>> call(Params params);
}

class NoParams extends Equatable {
  const NoParams();

  @override
```

```
  List<Object?> get props => [];
}
```

### حقن الاعتماديات وأعمار الكائنات

`initDependencies()` يسجّل الأساس (التخزين، `SessionExpiry` ، `ApiClient` )، ثم يستدعي دالة `_initX()` لكل ميزة. المصدر والمستودع والـ UseCases بلا حالة، فتُسجّل `registerLazySingleton` (نسخة واحدة عند أول طلب). الـ Cubit يحمل حالة شاشة ويُغلق معها، فيُسجّل `registerFactory` (نسخة جديدة لكل صفحة).

### الموجّه والجلسة

`createRouter()` يقرر الوجهة: بلا جلسة لا تُفتح إلا صفحات الدخول، ومع جلسة لا تُفتح صفحات الدخول. حين يرد الخادم بـ 401 على طلب بتوكن، يمسح `SessionExpiry` الجلسة ويُعلم الموجّه، فيعيد تقييم `redirect` ويفتح صفحة الدخول تلقائياً. معطيات المسار تُمرَّر في `pathParameters` أو `queryParameters` لا في `extra` ، حتى تعمل الصفحة إن فُتحت مباشرة.

### الثيم والنصوص والتحقق

| الملف | القاعدة |
|---|---|
| `AppStrings` | كل نص يظهر للمستخدم. لا نص عربي داخل ملف واجهة |
| `AppColors` | كل لون، باسم يصف دوره. لا `Color(0x…)` في أي ميزة |
| `app_icons.dart` | أيقونات Phosphor كخط: `PhosphorIconsRegular.x` و `PhosphorIconsFill.x` |
| `Validators` | قواعد الحقول المشتركة: `phone` ، `email` ، `notEmpty` ، `otp` . تعيد الرسالة أو `null` |

## 5. رحلة الخطأ

**Dio** — DioException: خطأ شبكة أو رد بحالة خطأ

↓

**ApiClient** — ServerException · ValidationException · AuthException

↓

**guard** — ServerFailure · ValidationFailure · AuthFailure

↓

**Cubit** — state.error: نص عربي جاهز للعرض

↓

**الواجهة** — ErrorView للجلب، أو SnackBar للأفعال

أي `Exception` آخر (من التخزين مثلاً) يصير «حدث خطأ غير متوقع». أما الخطأ البرمجي `Error` فلا يُبتلع، ليظهر لك أثناء التطوير.

الجزء الثاني

# المنهج: ست خطوات لكل ميزة

نبني من الداخل إلى الخارج. كل خطوة تنتهي بـ analyze نظيف، ومراجعة، وcommit.

| 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| **الفهم** | **Domain** | **Data** | **Cubit** | **الواجهة** | **التسليم** |
| التصميم والـ API | Entity وUseCases | Model وDataSource | الحالة والمنطق | DI والمسار والصفحة | تحقق ودمج |

**لماذا هذا الترتيب؟** كل طبقة تُبنى فوق شيء موجود ومُختبَر. إن بدأت من الواجهة ستكتب شاشة بلا بيانات، فتملؤها بكود مؤقت، وغالباً يبقى. والمثال في كل الخطوات ميزة `items` بعمليات CRUD الأربع: قراءة القائمة والعنصر، والإضافة، والتعديل، والحذف.

## الخطوة 0: الفهم قبل الكود

1. **من التصميم:** اكتب الشاشات، وما يفعله المستخدم في كل شاشة، والحالات الأربع لكل شاشة: تحميل، فارغ، خطأ، نجاح.
2. **لكل طلب في الخادم:** أرسله من Postman وسجّل شكل `data` داخل الغلاف، والحقول التي قد تأتي `null` أو بنوع آخر (رقم كنص مثلاً)، وردود الخطأ 400 و422 و404 و401. طلب فارغ يكشف قواعد الـ 422 دون أن يُحفظ شيء.
3. **احفظ رداً حقيقياً واحداً على الأقل**، ستضعه كما هو في اختبار البيانات.
4. **اسأل ما لا يقوله الرد:** بأي منطقة زمنية يُكتب الوقت؟ هل الجلب يغيّر شيئاً على الخادم؟ هل القائمة مرتبة؟ هل فيها صفحات؟
5. **افتح فرعاً** من `main` المحدّث: `git switch -c feat/<feature>`

سجّل ما وجدته في جدول كهذا. هذا جدول الإشعارات الحقيقي من هذه الدورة:

| حقل الخادم | مثال | في التطبيق | ملاحظة |
|---|---|---|---|
| `id` | `2` | `id: String` | رقم في الخادم، نص في التطبيق دائماً |
| `text` | `"test2"` | `body` | الـ Entity بلغة التطبيق، والـ Model يترجم |
| `created_at` | `"2026-10-04 15:52:33"` | `createdAt: DateTime` | بتوقيت UTC بلا علامة: تحققنا منه قبل الكتابة |
| `pivot.is_read` | `1` | `isRead: bool` | الجلب نفسه يجعله مقروءاً |
| `updated_at` ، بقية `pivot` | — | — | لا نحتاجها |

## الخطوة 1: Domain

Dart فقط، بلا Flutter ولا JSON. ثلاثة أنواع من الملفات: الكيان، والمستودع المجرد، والـ UseCases.

### الكيان: ما يراه التطبيق، وما يكتبه المستخدم

`Item` هو العنصر كما يصل من الخادم. `ItemDraft` هو ما يكتبه المستخدم في نموذج الإضافة أو التعديل: لا معرّف ولا تاريخ، لأن الخادم يحددهما. فصلهما يمنع إنشاء عنصر «نصف حقيقي» بمعرّف مؤقت.

lib/features/items/domain/entities/item.dart

```dart
import 'package:equatable/equatable.dart';

/// العنصر كما يراه التطبيق. استبدل `Item` باسم ميزتك: `Comment`، `Album`…
class Item extends Equatable {
  const Item({
    required this.id,
    required this.title,
    this.description,
    this.imageUrl,
    required this.createdAt,
  });

  final String id;
  final String title;

  /// اختياري: `null` إن لم يُكتب وصف.
  final String? description;
  final String? imageUrl;

  /// وقت الإنشاء. طريقة عرضه قرار الواجهة.
  final DateTime createdAt;

  @override
  List<Object?> get props => [id, title, description, imageUrl, createdAt];
}

/// ما يكتبه المستخدم عند الإضافة أو التعديل. لا `id` ولا تاريخ: الخادم يحددهما.
class ItemDraft extends Equatable {
  const ItemDraft({required this.title, this.description});

  final String title;
  final String? description;

  @override
  List<Object?> get props => [title, description];
}
```

### المستودع المجرد: عقد الميزة

كل دالة تعيد `Future<Either<Failure, T>>`. الدالتان الأخيرتان للحالات الإضافية في الجزء الثالث، احذفهما إن لم تحتجهما.

lib/features/items/domain/repositories/items_repository.dart

```dart
abstract class ItemsRepository {
  // القراءة (Read): القائمة، أو البحث فيها، أو عنصر واحد.
  Future<Either<Failure, List<Item>>> getItems({String? search});
```

```
  Future<Either<Failure, Item>> getItem(String id);

  // الكتابة (Create / Update / Delete).
  Future<Either<Failure, Item>> createItem(ItemDraft draft);
  Future<Either<Failure, Item>> updateItem(String id, ItemDraft draft);
  Future<Either<Failure, void>> deleteItem(String id);

  // حالات إضافية: صفحات، ورفع صورة.
  Future<Either<Failure, Paginated<Item>>> getItemsPage(int page);
  Future<Either<Failure, Item>> uploadImage(String id, ItemImage image);
}
```

## الـ UseCases: ملف لكل فعل

كلها بالشكل نفسه. هذه ثلاثة تمثل الأنواع الثلاثة للمعامل: بلا معامل ( `NoParams` )، ومعاملان معاً في كلاس `Params` ، وقيمة واحدة.

lib/features/items/domain/usecases/get_items.dart

```
class GetItems implements UseCase<List<Item>, NoParams> {
  const GetItems(this.repository);
  final ItemsRepository repository;

  @override
  Future<Either<Failure, List<Item>>> call(NoParams params) {
    return repository.getItems();
  }
}
```

lib/features/items/domain/usecases/update_item.dart

```
class UpdateItem implements UseCase<Item, UpdateItemParams> {
  const UpdateItem(this.repository);
  final ItemsRepository repository;

  @override
  Future<Either<Failure, Item>> call(UpdateItemParams params) {
    return repository.updateItem(params.id, params.draft);
  }
}

class UpdateItemParams extends Equatable {
  const UpdateItemParams({required this.id, required this.draft});

  final String id;
  final ItemDraft draft;

  @override
  List<Object?> get props => [id, draft];
}
```

lib/features/items/domain/usecases/delete_item.dart

```
class DeleteItem implements UseCase<void, String> {
  const DeleteItem(this.repository);
  final ItemsRepository repository;
```

```
  @override
  Future<Either<Failure, void>> call(String params) {
    return repository.deleteItem(params);
  }
}
```

الباقي بالنمط نفسه: `GetItem(String)` ، `CreateItem(ItemDraft)` ، `SearchItems(String)` ، `GetItemsPage(int)` ، `UploadItemImage(UploadItemImageParams)` .

### قواعد الخطوة 1

- كل الحقول `final` ، والمعرّفات `String` ، والاختياري nullable أو بقيمة افتراضية.
- `props` تضم **كل** الحقول، وإلا اعتُبر عنصران مختلفان متساويين.
- الوقت `DateTime` لا نص: «منذ ساعتين» قرار عرض يخص الواجهة.
- الـ UseCase بلا لاحقة: `GetItems` لا `GetItemsUseCase` .

commit

```
git commit -m "feat(items): add domain layer"
```

# الخطوة 2: Data

### الـ Model: ترجمة JSON إلى الكيان

يرث الـ Entity، وكل قراءة JSON تحدث هنا. `toJson` فقط إن كنت ستخزّن العنصر على الجهاز.

lib/features/items/data/models/item_model.dart

```
class ItemModel extends Item {
  const ItemModel({
    required super.id,
    required super.title,
    super.description,
    super.imageUrl,
    required super.createdAt,
  });

  factory ItemModel.fromJson(Map<String, dynamic> json) {
    return ItemModel(
      id: '${json['id']}',
      title: json['title'] as String,
      description: json['description'] as String?,
      imageUrl: json['image'] as String?,
      createdAt: _date(json['created_at'] as String),
    );
  }

  /// للتخزين المحلي فقط، بمفاتيح الخادم نفسها حتى يقرأها [ItemModel.fromJson].
  Map<String, dynamic> toJson() => {
        'id': id,
        'title': title,
        'description': description,
        'image': imageUrl,
        'created_at': createdAt.toUtc().toIso8601String().replaceFirst('Z', ''),
      };

  /// الخادم يكتب الوقت بتوقيت UTC بلا علامة المنطقة، فنضيفها ثم نحوّله لتوقيت الجهاز.
  static DateTime _date(String value) => DateTime.parse('${value}Z').toLocal();
}
```

> **صارم أم متسامح؟**
>
> الحقل الذي رأيته في رد حقيقي يُقرأ بصرامة ( `as String` ): إن غيّر الخادم شكله تظهر رسالة «رد غير متوقع» بدل شاشة متجمدة. الحقل الذي لم تره بعد يُقرأ بتسامح مع قيمة افتراضية وتعليق يقول ذلك، كما في `SubscriptionPackageModel` .

### الـ DataSource: الحديث مع الخادم

واجهة مجردة تعيد Models، وتنفيذ يستدعي `ApiClient` . المسار يبدأ بـ `/` ، و `parse:` في كل طلب يعيد بيانات، ولا `try/catch` . مسارات خادم درب كلها GET للقراءة وPOST للكتابة ( `add` ، `update` ، `delete` )، فالقالب يتبعها.

lib/features/items/data/datasources/items_remote_data_source.dart

```
abstract class ItemsRemoteDataSource {
  Future<List<ItemModel>> getItems({String? search});
```

```
  Future<ItemModel> getItem(String id);
  Future<ItemModel> createItem(ItemDraft draft);
  Future<ItemModel> updateItem(String id, ItemDraft draft);
  Future<void> deleteItem(String id);
  Future<Paginated<ItemModel>> getItemsPage(int page);
  Future<ItemModel> uploadImage(String id, ItemImage image, String contentType);
}

class ItemsRemoteDataSourceImpl implements ItemsRemoteDataSource {
  const ItemsRemoteDataSourceImpl(this._client);
  final ApiClient _client;

  @override
  Future<List<ItemModel>> getItems({String? search}) {
    return _client.get('/items/all', query: {'search': ?search}, parse: _list);
  }

  @override
  Future<ItemModel> getItem(String id) {
    return _client.get('/items/details', query: {'id': id}, parse: _item);
  }

  @override
  Future<ItemModel> createItem(ItemDraft draft) {
    return _client.post('/items/add', fields: _fields(draft), parse: _item);
  }

  @override
  Future<ItemModel> updateItem(String id, ItemDraft draft) {
    return _client.post('/items/update', fields: {'id': id, ..._fields(draft)}, parse: _item);
  }

  @override
  Future<void> deleteItem(String id) async {
    await _client.post('/items/delete', fields: {'id': id});
  }

  @override
  Future<Paginated<ItemModel>> getItemsPage(int page) {
    return _client.get(
      '/items/paginate',
      query: {'page': page},
      parse: (data) => parsePaginated(data, ItemModel.fromJson),
    );
  }

  @override
  Future<ItemModel> uploadImage(String id, ItemImage image, String contentType) {
    return _client.upload(
      '/items/update-image',
      field: 'image',
      bytes: image.bytes,
      fileName: image.fileName,
      contentType: contentType,
      fields: {'id': id},
      parse: _item,
    );
  }

  /// أسماء الحقول كما يطلبها الخادم. الحقل الفارغ لا يُرسل.
```

```
  static Map<String, dynamic> _fields(ItemDraft draft) => {
        'title': draft.title,
        'description': ?draft.description,
      };

  static ItemModel _item(dynamic data) => ItemModel.fromJson(data as Map<String, dynamic>);

  static List<ItemModel> _list(dynamic data) => [
        for (final json in (data as List).cast<Map<String, dynamic>>()) ItemModel.fromJson(json),
      ];
}
```

> **إن لم يجهز الطلب بعد**
>
> اكتب `ItemsMockDataSource implements ItemsRemoteDataSource` يقرأ من `MockCatalog` ، وسجّله في DI. حين يجهز الخادم تستبدل سطر التسجيل وحده، ولا يتغير المستودع ولا الـ Cubit.

## الـ RepositoryImpl: تحقق، نظّف، استدعِ داخل guard

lib/features/items/data/repositories/items_repository_impl.dart

```
class ItemsRepositoryImpl implements ItemsRepository {
  const ItemsRepositoryImpl(this._remote);
  final ItemsRemoteDataSource _remote;

  @override
  Future<Either<Failure, List<Item>>> getItems({String? search}) {
    return guard(() => _remote.getItems(search: search?.trim()));
  }

  @override
  Future<Either<Failure, Item>> getItem(String id) => guard(() => _remote.getItem(id));

  @override
  Future<Either<Failure, Item>> createItem(ItemDraft draft) async {
    final error = _validate(draft);
    if (error != null) return Left(ValidationFailure(error));
    return guard(() => _remote.createItem(_clean(draft)));
  }

  @override
  Future<Either<Failure, Item>> updateItem(String id, ItemDraft draft) async {
    final error = _validate(draft);
    if (error != null) return Left(ValidationFailure(error));
    return guard(() => _remote.updateItem(id, _clean(draft)));
  }

  @override
  Future<Either<Failure, void>> deleteItem(String id) => guard(() => _remote.deleteItem(id));

  @override
  Future<Either<Failure, Paginated<Item>>> getItemsPage(int page) => guard(() =>
_remote.getItemsPage(page));

  @override
  Future<Either<Failure, Item>> uploadImage(String id, ItemImage image) async {
    final contentType = image.contentType;
```

```
    if (contentType == null) return const Left(ValidationFailure(AppStrings.errorImageType));
    return guard(() => _remote.uploadImage(id, image, contentType));
  }

  /// قواعد الحقول. `null` يعني صالح.
  static String? _validate(ItemDraft draft) => Validators.notEmpty(draft.title);

  /// يزيل المسافات، والوصف الفارغ يصير `null` فلا يُرسل.
  static ItemDraft _clean(ItemDraft draft) {
    final description = draft.description?.trim();
    return ItemDraft(
      title: draft.title.trim(),
      description: description == null || description.isEmpty ? null : description,
    );
  }
}
```

## اختبار البيانات

`FakeHttpAdapter` يرد على كل مسار بما تحدده، ويسجّل الطلبات المرسلة. غطِّ: قراءة الرد الحقيقي، والمسار والحقول المرسلة، والرفض قبل الطلب، وخطأ 422.

test/features/items/items_data_test.dart

```
/// (0 ضع هنا رداً حقيقياً من الخطوة) عنصر كما يعيده الخادم.
const _json = {
  'id': 7,
  'title': 'عنوان',
  'description': 'وصف',
  'image': null,
  'created_at': '2026-10-04 15:52:33',
};

void main() {
  test('fromJson: يتحول لتوقيت الجهاز المعرّف UTC نص، والوقت', () {
    // Equatable يقارن النوع أيضاً، فالمتوقَّع Model لا Entity.
    expect(
      ItemModel.fromJson(_json),
      ItemModel(
        id: '7',
        title: 'عنوان',
        description: 'وصف',
        createdAt: DateTime.utc(2026, 10, 4, 15, 52, 33).toLocal(),
      ),
    );
  });

  test('toJson ثم fromJson (للتخزين المحلي) يعيد العنصر نفسه', () {
    final item = ItemModel.fromJson(_json);
    expect(ItemModel.fromJson(item.toJson()), item);
  });

  group('المصدر والمستودع', () {
    late FakeHttpAdapter adapter;
    late ItemsRepositoryImpl repository;

    setUp(() {
      adapter = FakeHttpAdapter();
      final dio = Dio(BaseOptions(baseUrl: 'https://api.test/api'))..httpClientAdapter = adapter;
```

```
      repository = ItemsRepositoryImpl(ItemsRemoteDataSourceImpl(ApiClient(readToken: () => '30|abc', dio:
dio)));
    });

    test('القراءة: القائمة من items/all، والعنصر من items/details?id=', () async {
      adapter
        ..on('/items/all', body: {'message': 'ok', 'data': [_json]})
        ..on('/items/details', body: {'message': 'ok', 'data': _json});

      expect((await repository.getItems()).getOrElse(() => []).single.title, 'عنوان');
      expect((await repository.getItem('7')).isRight(), isTrue);
      expect(adapter.requests.last.queryParameters, {'id': '7'});
    });

    test('الإضافة ترسل الحقول منظّفة، والوصف الفارغ لا يُرسل', () async {
      adapter.on('/items/add', body: {'message': 'ok', 'data': _json});

      await repository.createItem(const ItemDraft(title: '  عنوان  ', description: ' '));

      expect(formFields(adapter.requests.single), {'title': 'عنوان'});
    });

    test('التعديل يرسل المعرّف مع الحقول', () async {
      adapter.on('/items/update', body: {'message': 'ok', 'data': _json});

      await repository.updateItem('7', const ItemDraft(title: 'عنوان', description: 'وصف'));

      expect(formFields(adapter.requests.single), {'id': '7', 'title': 'عنوان', 'description': 'وصف'});
    });

    test('الحذف يرسل المعرّف إلى items/delete', () async {
      adapter.on('/items/delete', body: {'message': 'تم الحذف', 'data': []});

      expect(await repository.deleteItem('7'), const Right<Failure, void>(null));
      expect(formFields(adapter.requests.single), {'id': '7'});
    });

    test('عنوان فارغ يُرفض قبل أي طلب', () async {
      final result = await repository.createItem(const ItemDraft(title: ' '));

      expect(result, const Left<Failure, Item>(ValidationFailure(AppStrings.errorFieldsRequired)));
      expect(adapter.requests, isEmpty);
    });
    // … نفسه الملف في والصفحات، 422، ورد.
  });
}
```

### Equatable يقارن النوع

`ItemModel(...)` لا يساوي `Item(...)` ولو تطابقت الحقول. في اختبار البيانات قارن دائماً بالـ Model.

commit

```
git commit -m "feat(items): add data layer"
```

## الخطوة 3: Cubit وState

Cubit لكل شاشة، أو لكل جزء يُفتح وحده (ورقة، نافذة). الحالة في ملف `part` مستقل. ميزة CRUD تحتاج ثلاثة: القائمة، والتفاصيل، والنموذج.

| نوع الحالة | القيم | متى |
|---|---|---|
| جلب بيانات | `initial, loading, success, failure` | شاشة تعرض بيانات من الخادم |
| فعل يرسل شيئاً | `idle, sending, done, failure` | حفظ، حذف، تفعيل. في حقل منفصل عن حالة الجلب |

### القائمة: الجلب والحذف والتحديث بالسحب

lib/features/items/presentation/cubit/items_state.dart

```dart
part of 'items_cubit.dart';

enum ItemsStatus { initial, loading, success, failure }

/// حالة فعل على القائمة (الحذف، أو تحديث بالسحب).
enum ItemActionStatus { idle, sending, done, failure }

class ItemsState extends Equatable {
  const ItemsState({
    this.status = ItemsStatus.initial,
    this.items = const [],
    this.error,
    this.action = ItemActionStatus.idle,
    this.actionError,
  });

  final ItemsStatus status;
  final List<Item> items;

  /// يُعرض فقط مع `failure`.
  final String? error;

  /// منفصلة عن حالة الجلب: فشل الحذف لا يمسح القائمة.
  final ItemActionStatus action;
  final String? actionError;

  ItemsState copyWith({
    ItemsStatus? status,
    List<Item>? items,
    String? error,
    ItemActionStatus? action,
    String? actionError,
  }) {
    return ItemsState(
      status: status ?? this.status,
      items: items ?? this.items,
      error: error,
      action: action ?? this.action,
      actionError: actionError,
    );
  }
```

```
  @override
  List<Object?> get props => [status, items, error, action, actionError];
}
```

lib/features/items/presentation/cubit/items_cubit.dart

```
class ItemsCubit extends Cubit<ItemsState> {
  ItemsCubit({required this._getItems, required this._deleteItem}) : super(const ItemsState());

  final GetItems _getItems;
  final DeleteItem _deleteItem;

  Future<void> load() async {
    emit(state.copyWith(status: ItemsStatus.loading));
    final result = await _getItems(const NoParams());
    if (isClosed) return;
    emit(
      result.fold(
        (failure) => state.copyWith(status: ItemsStatus.failure, error: failure.message),
        (items) => state.copyWith(status: ItemsStatus.success, items: items),
      ),
    );
  }

  /// سحب للتحديث: تبقى القائمة معروضة، والفشل رسالة لا شاشة خطأ.
  Future<void> refresh() async {
    final result = await _getItems(const NoParams());
    if (isClosed) return;
    emit(
      result.fold(
        (failure) => state.copyWith(action: ItemActionStatus.failure, actionError: failure.message),
        (items) => state.copyWith(status: ItemsStatus.success, items: items),
      ),
    );
  }

  Future<void> delete(String id) async {
    emit(state.copyWith(action: ItemActionStatus.sending));
    final result = await _deleteItem(id);
    if (isClosed) return;
    emit(
      result.fold(
        (failure) => state.copyWith(action: ItemActionStatus.failure, actionError: failure.message),
        (_) => state.copyWith(
          action: ItemActionStatus.done,
          items: [for (final item in state.items) if (item.id != id) item],
        ),
      ),
    );
  }

  /// العنصر الذي حفظته ورقة الإضافة أو التعديل: يحل محل القديم بمعرّفه، أو يُضاف أولاً.
  void saved(Item item) {
    final exists = state.items.any((i) => i.id == item.id);
    emit(
      state.copyWith(
        items: exists ? [for (final i in state.items) i.id == item.id ? item : i] : [item,
...state.items],
      ),
    );
```

```
  }
}
```

**ثلاث قواعد في هذا الملف**

- `if (isClosed) return;` بعد كل `await` : قد تُغلق الصفحة قبل وصول الرد، والإصدار في Cubit مغلق خطأ.
- **الفعل في حقل منفصل:** فشل الحذف لا يمسح القائمة، بل يظهر رسالة فوقها.
- **لا إعادة جلب بلا داعٍ:** بعد الحذف أو الحفظ تُعدَّل القائمة محلياً بقائمة جديدة (لا تعديل على القديمة).

## النموذج: إضافة وتعديل بورقة واحدة

lib/features/items/presentation/cubit/item_form_state.dart

```
enum ItemFormStatus { idle, sending, done, failure }

class ItemFormState extends Equatable {
  const ItemFormState({this.status = ItemFormStatus.idle, this.saved, this.error});

  final ItemFormStatus status;

  /// العنصر كما أعاده الخادم بعد الحفظ، لتعيده الورقة إلى القائمة.
  final Item? saved;
  final String? error;

  @override
  List<Object?> get props => [status, saved, error];
}
```

lib/features/items/presentation/cubit/item_form_cubit.dart

```
/// ورقة واحدة للإضافة والتعديل: بلا `id` إضافة، ومعه تعديل.
class ItemFormCubit extends Cubit<ItemFormState> {
  ItemFormCubit({required this._createItem, required this._updateItem}) : super(const ItemFormState());

  final CreateItem _createItem;
  final UpdateItem _updateItem;

  Future<void> submit(ItemDraft draft, {String? id}) async {
    emit(const ItemFormState(status: ItemFormStatus.sending));
    final result = id == null ? await _createItem(draft) : await _updateItem(UpdateItemParams(id: id,
draft: draft));
    if (isClosed) return;
    emit(
      result.fold(
        (failure) => ItemFormState(status: ItemFormStatus.failure, error: failure.message),
        (item) => ItemFormState(status: ItemFormStatus.done, saved: item),
      ),
    );
  }
}
```

### التفاصيل: قراءة عنصر واحد

lib/features/items/presentation/cubit/item_details_cubit.dart

```
class ItemDetailsCubit extends Cubit<ItemDetailsState> {
  ItemDetailsCubit(this._getItem) : super(const ItemDetailsState());

  final GetItem _getItem;

  Future<void> load(String id) async {
    emit(const ItemDetailsState(status: ItemDetailsStatus.loading));
    final result = await _getItem(id);
    if (isClosed) return;
    emit(
      result.fold(
        (failure) => ItemDetailsState(status: ItemDetailsStatus.failure, error: failure.message),
        (item) => ItemDetailsState(status: ItemDetailsStatus.success, item: item),
      ),
    );
  }
}
```

### اختبار الـ Cubit

مستودع مزيّف `implements` الواجهة، والـ UseCases حقيقية. هكذا تختبر المنطق كله بلا خادم.

test/features/items/items_cubits_test.dart

```
ItemsCubit _list(_FakeRepository repository) =>
    ItemsCubit(getItems: GetItems(repository), deleteItem: DeleteItem(repository));

void main() {
  group('ItemsCubit', () {
    test('يجلب القائمة', () async {
      final cubit = _list(_FakeRepository());
      await cubit.load();

      expect(cubit.state, ItemsState(status: ItemsStatus.success, items: [_a, _b]));
    });

    test('الحذف يزيل العنصر من القائمة دون إعادة جلبها', () async {
      final cubit = _list(_FakeRepository());
      await cubit.load();
      await cubit.delete('1');

      expect(cubit.state.items, [_b]);
      expect(cubit.state.action, ItemActionStatus.done);
```

> **تمرين من الدورة**
>
> احذف مؤقتاً سطر `if (isClosed) return;` وشغّل اختبار «إغلاق الصفحة قبل وصول الرد» في الإشعارات: سيفشل بـ `StateError` . الاختبار الذي لم تره يفشل مرة لا تعرف إن كان يختبر شيئاً.

commit

```
git commit -m "test(items): cover the items cubits"
```

# الخطوة 4: DI والمسار والواجهة

## التسجيل في DI

lib/core/di/injection.dart، واستدعِها من initDependencies()

```
void _initItems() {
  sl
    ..registerLazySingleton<ItemsRemoteDataSource>(() => ItemsRemoteDataSourceImpl(sl()))
    ..registerLazySingleton<ItemsRepository>(() => ItemsRepositoryImpl(sl()))
    ..registerLazySingleton(() => GetItems(sl()))
    ..registerLazySingleton(() => GetItem(sl()))
    ..registerLazySingleton(() => CreateItem(sl()))
    ..registerLazySingleton(() => UpdateItem(sl()))
    ..registerLazySingleton(() => DeleteItem(sl()))
    ..registerFactory(() => ItemsCubit(getItems: sl(), deleteItem: sl()))
    ..registerFactory(() => ItemDetailsCubit(sl()))
    ..registerFactory(() => ItemFormCubit(createItem: sl(), updateItem: sl()));
}
```

## المسارات

lib/core/router/app_router.dart

```
GoRoute(
  path: '/items',
  builder: (_, _) => BlocProvider(
    create: (_) => sl<ItemsCubit>()..load(),
    child: const ItemsPage(),
  ),
),
GoRoute(
  path: '/items/:id',
  builder: (context, state) {
    final id = state.pathParameters['id']!;
    return BlocProvider(
      create: (_) => sl<ItemDetailsCubit>()..load(id),
      child: ItemDetailsPage(itemId: id),
    );
  },
),
```

## ما تضيفه إلى core مرة واحدة

رسالة القائمة الفارغة تتكرر في كل ميزة، فمكانها `core` بجانب `ErrorView` (القاعدة: ما تحتاجه ميزتان يُنقل إلى core):

lib/core/widgets/error_view.dart (مع import app_icons.dart)

```
/// رسالة القائمة الفارغة بشكل [ErrorView]، نفسه، بلا زر.
class EmptyView extends StatelessWidget {
  const EmptyView({super.key, required this.message, this.icon = PhosphorIconsRegular.listBullets});

  final String message;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
```

```
    return Padding(
      padding: const EdgeInsets.fromLTRB(24, 70, 24, 24),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(icon, color: AppColors.textMuted, size: 48),
          const SizedBox(height: 12),
          Text(
            message,
            textAlign: TextAlign.center,
            style: const TextStyle(color: AppColors.textSecondary, fontSize: 14),
          ),
        ],
      ),
    );
  }
}
```

app_strings.dart و app_icons.dart

```
// AppStrings
static const items = 'العناصر';
static const itemDetails = 'تفاصيل العنصر';
static const noItems = 'لا توجد عناصر بعد';
static const addItem = 'إضافة عنصر';
static const editItem = 'تعديل العنصر';
static const deleteItem = 'حذف العنصر';
static const deleteItemConfirm = 'هل تريد حذف هذا العنصر؟';
static const itemDeleted = 'تم حذف العنصر';
static const itemTitleHint = 'العنوان';
static const itemDescriptionHint = 'الوصف (اختياري)';

// PhosphorIconsRegular
static const plus = IconData(0xe3d4, fontFamily: _family);
static const trash = IconData(0xe4a6, fontFamily: _family);
static const pencilSimple = IconData(0xe3b4, fontFamily: _family);
```

## صفحة القائمة

lib/features/items/presentation/pages/items_page.dart

```
class ItemsPage extends StatelessWidget {
  const ItemsPage({super.key});

  /// يُقرأ الـ Cubit قبل `await`، لأن `context` قد لا يبقى صالحاً بعده.
  Future<void> _openForm(BuildContext context, {Item? item}) async {
    final cubit = context.read<ItemsCubit>();
    final saved = await showItemForm(context, item: item);
    if (saved != null) cubit.saved(saved);
  }

  Future<void> _delete(BuildContext context, Item item) async {
    final cubit = context.read<ItemsCubit>();
    if (await showConfirmDialog(context, title: AppStrings.deleteItemConfirm)) cubit.delete(item.id);
  }

  void _showMessage(BuildContext context, String message) {
    ScaffoldMessenger.of(context)
      ..hideCurrentSnackBar()
```

```
      ..showSnackBar(SnackBar(content: Text(message)));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      floatingActionButton: FloatingActionButton(
        backgroundColor: AppColors.primary,
        tooltip: AppStrings.addItem,
        onPressed: () => _openForm(context),
        child: const Icon(PhosphorIconsRegular.plus, color: Colors.white),
      ),
      body: Column(
        children: [
          PageHeader(
            title: AppStrings.items,
            onBack: () => context.pop(),
            onNotifications: () => context.push('/notifications'),
          ),
          Expanded(
            child: BlocConsumer<ItemsCubit, ItemsState>(
              // listener، ما يحدث مرة واحدة (رسالة) في بناء، ولا يتكرر مع كل بناء.
              listenWhen: (previous, current) => previous.action != current.action,
              listener: (context, state) {
                switch (state.action) {
                  case ItemActionStatus.done:
                    _showMessage(context, AppStrings.itemDeleted);
                  case ItemActionStatus.failure:
                    _showMessage(context, state.actionError ?? AppStrings.loadError);
                  case ItemActionStatus.idle || ItemActionStatus.sending:
                    break;
                }
              },
              builder: (context, state) {
                final cubit = context.read<ItemsCubit>();
                return switch (state.status) {
                  ItemsStatus.failure => ErrorView(message: state.error, onRetry: cubit.load),
                  ItemsStatus.success when state.items.isEmpty => const EmptyView(message:
AppStrings.noItems),
                  ItemsStatus.success => RefreshIndicator(
                      color: AppColors.primary,
                      onRefresh: cubit.refresh,
                      child: ListView.separated(
                        padding: const EdgeInsets.only(top: 8, bottom: 88),
                        itemCount: state.items.length,
                        separatorBuilder: (_, _) => const Divider(height: 1, color: AppColors.divider),
                        itemBuilder: (context, i) => _ItemTile(
                          item: state.items[i],
                          onEdit: () => _openForm(context, item: state.items[i]),
                          onDelete: () => _delete(context, state.items[i]),
                        ),
                      ),
                    ),
                  _ => const LoadingView(),
                };
              },
            ),
          ),
        ],
```

```
      ),
    );
  }
}
```

والسطر الواحد في القائمة …

```
class _ItemTile extends StatelessWidget {
  const _ItemTile({required this.item, required this.onEdit, required this.onDelete});

  final Item item;
  final VoidCallback onEdit;
  final VoidCallback onDelete;

  @override
  Widget build(BuildContext context) {
    final description = item.description;
    return ListTile(
      title: Text(item.title, style: const TextStyle(color: AppColors.blueStrong, fontWeight:
FontWeight.w500)),
      subtitle: description == null ? null : Text(description, maxLines: 2, overflow:
TextOverflow.ellipsis),
      onTap: () => context.push('/items/${item.id}'),
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          IconButton(
            tooltip: AppStrings.editItem,
            onPressed: onEdit,
            icon: const Icon(PhosphorIconsRegular.pencilSimple, color: AppColors.iconBlue),
          ),
          IconButton(
            tooltip: AppStrings.deleteItem,
            onPressed: onDelete,
            icon: const Icon(PhosphorIconsRegular.trash, color: AppColors.danger),
          ),
        ],
      ),
    );
  }
}
```

### قواعد الصفحة

- `switch` على `status` : تحميل، خطأ مع إعادة المحاولة، فارغ، نجاح. لا شاشة بيضاء أبداً.
- ما يحدث مرة واحدة (رسالة، رجوع، انتقال) في `listener` مع `listenWhen` ، لا في `builder` .
- اقرأ الـ Cubit قبل `await` ، لأن `context` قد لا يبقى صالحاً بعده.

## ورقة الإضافة والتعديل

lib/features/items/presentation/widgets/item_form_sheet.dart

```
/// ورقة الإضافة (بلا [item]) أو التعديل. تعيد العنصر المحفوظ، أو `null` إن أُغلقت.
Future<Item?> showItemForm(BuildContext context, {Item? item}) {
  return showAppSheet<Item>(
    context,
```

```
    (_) => BlocProvider(create: (_) => sl<ItemFormCubit>(), child: ItemFormSheet(item: item)),
  );
}

class ItemFormSheet extends StatefulWidget {
  const ItemFormSheet({super.key, this.item});

  final Item? item;

  @override
  State<ItemFormSheet> createState() => _ItemFormSheetState();
}

class _ItemFormSheetState extends State<ItemFormSheet> {
  late final _title = TextEditingController(text: widget.item?.title);
  late final _description = TextEditingController(text: widget.item?.description);

  @override
  void dispose() {
    _title.dispose();
    _description.dispose();
    super.dispose();
  }

  void _submit() {
    context.read<ItemFormCubit>().submit(
          ItemDraft(title: _title.text, description: _description.text),
          id: widget.item?.id,
        );
  }

  @override
  Widget build(BuildContext context) {
    return BlocConsumer<ItemFormCubit, ItemFormState>(
      listener: (context, state) {
        if (state.status == ItemFormStatus.done) Navigator.of(context).pop(state.saved);
      },
      builder: (context, state) {
        final error = state.error;
        return Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            SheetHeader(
              title: widget.item == null ? AppStrings.addItem : AppStrings.editItem,
              icon: PhosphorIconsRegular.notePencil,
            ),
            const SizedBox(height: 24),
            SheetField(hint: AppStrings.itemTitleHint, icon: PhosphorIconsRegular.notePencil, controller:
_title),
            const SizedBox(height: 24),
            SheetField(
              hint: AppStrings.itemDescriptionHint,
              icon: PhosphorIconsRegular.listBullets,
              controller: _description,
              maxLines: 4,
            ),
            if (state.status == ItemFormStatus.failure && error != null) ...[
              const SizedBox(height: 12),
              Text(error, style: const TextStyle(color: AppColors.danger, fontSize: 13)),
            ],
```

```
            const SizedBox(height: 24),
            SheetActions(
              primaryLabel: AppStrings.confirm,
              onPrimary: _submit,
              loading: state.status == ItemFormStatus.sending,
            ),
          ],
        );
      },
    );
  }
}
```

## صفحة التفاصيل

lib/features/items/presentation/pages/item_details_page.dart

```
            child: BlocBuilder<ItemDetailsCubit, ItemDetailsState>(
              builder: (context, state) => switch ((state.status, state.item)) {
                (ItemDetailsStatus.failure, _) => ErrorView(
                    message: state.error,
                    onRetry: () => context.read<ItemDetailsCubit>().load(itemId),
                  ),
                (ItemDetailsStatus.success, final Item item) => _Details(item: item),
                _ => const LoadingView(),
```

لاحظ `switch ((state.status, state.item))`: مطابقة على زوج تعطيك `item` غير nullable داخل حالة النجاح، بلا `!`.

## اختبار الواجهة

test/features/items/items_flow_test.dart

```
  testWidgets('الإضافة من الورقة تظهر في القائمة', (tester) async {
    await pumpPage(tester);

    await tester.tap(find.byTooltip(AppStrings.addItem));
    await tester.pumpAndSettle();
    await tester.enterText(find.byType(TextField).first, 'الثاني');
    await tester.tap(find.text(AppStrings.confirm));
    await tester.pumpAndSettle();

    expect(find.text('الأول'), findsOneWidget);
    expect(find.text('الثاني'), findsOneWidget);
  });

  testWidgets('الحذف بعد التأكيد يزيل العنصر، ويظهر رسالة', (tester) async {
    await pumpPage(tester);

    await tester.tap(find.byTooltip(AppStrings.deleteItem));
    await tester.pumpAndSettle();
    await tester.tap(find.text(AppStrings.confirm));
    await tester.pumpAndSettle();

    expect(find.text('الأول'), findsNothing);
    expect(find.text(AppStrings.noItems), findsOneWidget);
    expect(find.text(AppStrings.itemDeleted), findsOneWidget);
```

**لا تنسَ اختبار المسارات**

`routes_smoke_test` يستبدل الشبكة بمحوّل مزيّف ويتوقع ألا تظهر `ErrorView`. كل طلب جديد تطلبه صفحة يحتاج رداً هناك، مثل: `..on('/items/all', body: {'message': 'ok', 'data': []})`، وأضف المسار إلى قائمة `paths`.

commit

```
git commit -m "feat(items): build the items screens"
```

## الخطوة 5: التحقق والتسليم

قبل كل commit

```
flutter analyze
flutter test
git diff main --stat
```

1. **على الخادم الحقيقي** (رمز التحقق 1111): جرّب الحالات الأربع، وافصل الإنترنت لترى رسالة الخطأ وزر إعادة المحاولة.
2. **راجع الفرق:** لا `print` ، ولا `TODO` ، ولا كود معلّق، ولا ملفات شخصية.
3. **ادمج:** إن كان `main` لم يتحرك منذ أنشأت الفرع، فالدمج مباشر (fast-forward) ويبقى التاريخ مستقيماً.

الدمج والرفع

```
git switch main
git merge --ff-only feat/items
git push origin main feat/items
```

| النوع | متى | مثال |
|---|---|---|
| `feat` | ميزة أو جزء منها | `feat(items): add data layer` |
| `fix` | إصلاح خطأ | `fix(core): handle unexpected responses` |
| `refactor` | إعادة ترتيب بلا تغيير سلوك | `refactor: unify feature conventions` |
| `test` | اختبارات فقط | `test(notifications): cover NotificationsCubit` |
| `docs` | توثيق وتعليقات | `docs(notifications): note that fetching marks …` |

الجزء الثالث

# حالات استخدام إضافية

ما يتجاوز CRUD البسيط: البحث، والقوائم الطويلة، ورفع الملفات، والعمل بلا إنترنت.

## البحث أثناء الكتابة

المشكلة: طلب مع كل حرف يرهق الخادم، والردود قد تصل بغير ترتيبها. الحل: انتظر توقف الكتابة (debounce) 400ms، وأهمِل أي رد لبحث لم يعد الحالي.

lib/features/items/presentation/cubit/items_search_cubit.dart

```
class ItemsSearchCubit extends Cubit<ItemsSearchState> {
  ItemsSearchCubit(this._searchItems) : super(const ItemsSearchState());

  final SearchItems _searchItems;
  Timer? _debounce;

  void queryChanged(String query) {
    _debounce?.cancel();
    final trimmed = query.trim();
    if (trimmed.isEmpty) {
      emit(const ItemsSearchState());
      return;
    }
    _debounce = Timer(const Duration(milliseconds: 400), () => _search(trimmed));
  }

  Future<void> _search(String query) async {
    emit(ItemsSearchState(status: ItemsSearchStatus.loading, query: query, results: state.results));
    final result = await _searchItems(query);
    // أُغلقت الصفحة، أو غيّر المستخدم البحث قبل وصول هذا الرد.
    if (isClosed || query != state.query) return;
    emit(
      result.fold(
        (failure) => ItemsSearchState(status: ItemsSearchStatus.failure, query: query, error:
failure.message),
        (items) => ItemsSearchState(status: ItemsSearchStatus.success, query: query, results: items),
      ),
    );
  }

  @override
  Future<void> close() {
    _debounce?.cancel();
    return super.close();
```

```
  }
}
```

في الصفحة

```
TextField(
  onChanged: context.read<ItemsSearchCubit>().queryChanged,
  decoration: const InputDecoration(hintText: AppStrings.search),
)
```

## القوائم الطويلة: صفحةً صفحة

الخادم يقدّم `*/paginate` (الأخبار، الدورات، المعرض...). النوع العام في `core` لأن كل الميزات تحتاجه، وقراءة شكل Laravel في دالة واحدة:

lib/core/pagination/paginated.dart

```
class Paginated<T> extends Equatable {
  const Paginated({required this.items, required this.page, required this.hasMore});

  final List<T> items;
  final int page;
  final bool hasMore;

  @override
  List<Object?> get props => [items, page, hasMore];
}
```

lib/core/pagination/paginated_json.dart

```
/// يقرأ صفحة Laravel داخل `data`: `{current_page, last_page, data: [...]}`.
/// تحقق من الشكل في الخطوة 0 قبل الاعتماد عليه.
Paginated<T> parsePaginated<T>(dynamic data, T Function(Map<String, dynamic> json) item) {
  final json = data as Map<String, dynamic>;
  final page = json['current_page'] as int;
  return Paginated(
    items: [for (final element in (json['data'] as List).cast<Map<String, dynamic>>()) item(element)],
    page: page,
    hasMore: page < (json['last_page'] as int),
  );
}
```

lib/features/items/presentation/cubit/paged_items_cubit.dart

```
class PagedItemsCubit extends Cubit<PagedItemsState> {
  PagedItemsCubit(this._getItemsPage) : super(const PagedItemsState());

  final GetItemsPage _getItemsPage;

  Future<void> loadFirst() async {
    emit(const PagedItemsState(status: PagedItemsStatus.loading));
    await _load(1);
  }

  /// عند الاقتراب من نهاية القائمة. لا يرسل طلباً ثانياً قبل عودة الأول.
```

```
  Future<void> loadMore() async {
    if (state.status != PagedItemsStatus.success || !state.hasMore || state.loadingMore) return;
    emit(state.copyWith(loadingMore: true));
    await _load(state.page + 1);
  }

  Future<void> _load(int page) async {
    final result = await _getItemsPage(page);
    if (isClosed) return;
    emit(
      result.fold(
        // فشل الصفحة الأولى شاشة خطأ، وفشل ما بعدها رسالة تحت القائمة.
        (failure) => page == 1
            ? PagedItemsState(status: PagedItemsStatus.failure, error: failure.message)
            : state.copyWith(loadingMore: false, moreError: failure.message),
        (paged) => state.copyWith(
          status: PagedItemsStatus.success,
          items: [...state.items, ...paged.items],
          page: paged.page,
          hasMore: paged.hasMore,
          loadingMore: false,
        ),
      ),
    );
  }
}
```

lib/features/items/presentation/widgets/paged_items_list.dart

```
          PagedItemsStatus.success => NotificationListener<ScrollNotification>(
              onNotification: (notification) {
                if (notification.metrics.extentAfter < 300) cubit.loadMore();
                return false;
              },
              child: ListView.builder(
                // عنصر أخير للتحميل أو لخطأ الصفحة التالية ما دام هناك المزيد.
                itemCount: state.items.length + (state.hasMore ? 1 : 0),
                itemBuilder: (context, i) {
                  if (i < state.items.length) return ListTile(title: Text(state.items[i].title));
                  final moreError = state.moreError;
                  return moreError == null
                      ? const Padding(padding: EdgeInsets.all(16), child: LoadingView())
                      : TextButton(onPressed: cubit.loadMore, child: Text('$moreError —
${AppStrings.retry}'));
                },
              ),
            ),
          _ => const LoadingView(),
```

## رفع صورة

الكيان على نمط `ProfileImage` : البايتات واسم الملف، ونوعه من امتداده.

lib/features/items/domain/entities/item_image.dart

```
class ItemImage extends Equatable {
  const ItemImage({required this.bytes, required this.fileName});
```

```
  final Uint8List bytes;
  final String fileName;

  static const allowedTypes = {'jpg': 'image/jpeg', 'jpeg': 'image/jpeg', 'png': 'image/png'};

  /// نوع الملف من امتداده، أو `null` إن لم يكن مقبولاً.
  String? get contentType {
    final dot = fileName.lastIndexOf('.');
    if (dot == -1) return null;
    return allowedTypes[fileName.substring(dot + 1).toLowerCase()];
  }

  @override
  List<Object?> get props => [bytes, fileName];
}
```

رفع صورة لعنصر يحتاج إرسال `id` معها، و`ApiClient.upload` اليوم يرسل الملف وحده. أضف له معامل `fields` (تعديل صغير في core، يعمل مع كل الاستدعاءات الحالية):

lib/core/network/api_client.dart

```
Future<T> upload<T>(
  String path, {
  required String field,
  required List<int> bytes,
  required String fileName,
  required String contentType,
  Map<String, dynamic>? fields,          // جديد
  T Function(dynamic data)? parse,
}) {
  return _send(
    () => _dio.post<Object?>(
      path,
      data: FormData.fromMap({
        ...?fields,                      // جديد
        field: MultipartFile.fromBytes(
          bytes,
          filename: fileName,
          contentType: DioMediaType.parse(contentType),
        ),
      }),
    ),
    parse,
  );
}
```

المستودع: النوع يُتحقق منه قبل الرفع

```
  Future<Either<Failure, Item>> uploadImage(String id, ItemImage image) async {
    final contentType = image.contentType;
    if (contentType == null) return const Left(ValidationFailure(AppStrings.errorImageType));
    return guard(() => _remote.uploadImage(id, image, contentType));
  }
```

## العمل بلا إنترنت: تخزين آخر قائمة

على نمط `AuthLocalDataSource` : مصدر محلي بـ `SharedPreferences` ، يحفظ آخر قائمة بـ `toJson` ويقرأها بـ `fromJson` .

lib/features/items/data/datasources/items_local_data_source.dart

```
/// آخر قائمة وصلت من الخادم، لعرضها حين ينقطع الاتصال.
abstract class ItemsLocalDataSource {
  List<ItemModel>? get cachedItems;
  Future<void> saveItems(List<ItemModel> items);
}

class SharedPrefsItemsLocalDataSource implements ItemsLocalDataSource {
  SharedPrefsItemsLocalDataSource(this._prefs);

  static const _key = 'items.cache';
  final SharedPreferences _prefs;

  @override
  List<ItemModel>? get cachedItems {
    final raw = _prefs.getString(_key);
    if (raw == null) return null;
    try {
      return [
        for (final json in (jsonDecode(raw) as List).cast<Map<String, dynamic>>())
ItemModel.fromJson(json),
      ];
    } catch (_) {
      // نسخة تالفة أو بصيغة قديمة: كأنها غير موجودة.
      return null;
    }
  }

  @override
  Future<void> saveItems(List<ItemModel> items) async {
    await _prefs.setString(_key, jsonEncode([for (final item in items) item.toJson()]));
  }
}
```

المستودع: الشبكة أولاً، والنسخة المحفوظة عند الفشل

```
  /// الشبكة أولاً، والنسخة المحفوظة عند الفشل. البحث لا يُحفظ.
  @override
  Future<Either<Failure, List<Item>>> getItems({String? search}) async {
    final result = await guard(() async {
      final items = await _remote.getItems(search: search);
      if (search == null) await _local.saveItems(items);
      return items;
    });
    if (result.isRight() || search != null) return result;
    final cached = _local.cachedItems;
    return cached == null ? result : Right(cached);
  }
```

## تحويل ميزة تجريبية إلى الخادم

هذا ما تعلمناه حين حوّلنا الإشعارات. الترتيب نفسه، مع أربعة فروق:

1. **تغيير الـ Entity يكسر أماكن تستخدمه** (البيانات التجريبية والصفحة). أصلحها بأقل تعديل في الخطوة نفسها، والمحلّل يدلك عليها كلها.
2. **احذف المصدر التجريبي وبدّل تسجيله في DI في الخطوة 2 نفسها**، وإلا لن يُبنى المشروع.
3. **احذف بياناته من** `MockCatalog` ، حذفاً لا تعليقاً.
4. **أضف رد الطلب الجديد إلى** `routes_smoke_test` ، وإلا فشل اختبار المسارات.

### إن احتجت PUT أو DELETE

خادم درب لا يستخدمهما، لكن إن صادفتهما في مشروع آخر فأضف دالة بالنمط نفسه في `ApiClient` :

```
Future<T> delete<T>(String path, {Map<String, dynamic>? query, T Function(dynamic data)? parse}) {
  return _send(() => _dio.delete<Object?>(path, queryParameters: query), parse);
}
```

الجزء الرابع

# المرجع السريع

ما تراجعه في كل PR، في صفحات قليلة.

## قواعد الاعتماد

| الطبقة | تستورد من | ممنوع عليها |
|---|---|---|
| domain | domain، و `core` (error، usecase، utils، pagination) | data، presentation، Flutter |
| data | domain، data، core | presentation |
| presentation | domain، presentation، core | data |

- من ميزة أخرى: ملفات `domain` ، أو عنصر واجهة معدّ للمشاركة ( `showContactSheet` ). أما `data` ميزة أخرى فلا أبداً.
- ما تحتاجه ميزتان ولا ينتمي لإحداهما ينتقل إلى `core` (كما انتقل `Validators` و `EmptyView` ).
- `sl` في الواجهة لإنشاء Cubit فقط. الاستثناء الوحيد: الموجّه وشاشة البداية يقرآن `AuthRepository.cachedUser` .

## الأسماء

| العنصر | القاعدة | مثال |
|---|---|---|
| Entity | اسم الشيء | `Item` ، `SubscriptionPackage` |
| ما يكتبه المستخدم | الاسم + `Draft` | `ItemDraft` |
| Model | الـ Entity + `Model` | `ItemModel` |
| Repository | `<Feature>Repository` و `…Impl` | `ItemsRepositoryImpl` |
| DataSource | `<Feature>RemoteDataSource` و `…Impl` و `…MockDataSource` | `ItemsRemoteDataSourceImpl` |
| UseCase | فعل + مفعول، بلا لاحقة | `GetItems` ، `DeleteItem` |
| معامل مركّب | `<UseCase>Params` | `UpdateItemParams` |
| Cubit / State | `<Screen>Cubit` و `<Screen>State` في ملف part | `ItemFormCubit` |
| الفرع | `feat/` ، `fix/` ، `refactor/` | `feat/items-backend` |
| ملفات الاختبار | `_data_test` ، `_cubits_test` ، `_flow_test` | `items_flow_test.dart` |

## قائمة الانتهاء

- [ ] الحالات الأربع ظاهرة: تحميل، فارغ، خطأ مع إعادة المحاولة، نجاح
- [ ] ملفات الاختبار الثلاثة ناجحة، و `flutter analyze` نظيف، و `flutter test` كاملاً ناجح
- [ ] لا نص ولا لون مكتوب مباشرة في الواجهة
- [ ] `_initX()` في DI، والمسار في الموجّه وفي `routes_smoke_test`
- [ ] جُرّبت على الخادم الحقيقي
- [ ] لا TODO، ولا كود معلّق، ولا ملفات شخصية في الفرق
- [ ] ما عُرف عن الـ API مكتوب في تعليق الـ Model أو في الاختبار

## أخطاء صادفناها، ودرس كل منها

| ما حدث | أين | القاعدة |
|---|---|---|
| الاختبار نجح والكود خطأ | قراءة `created_at` كتوقيت محلي وهو UTC | الاختبار يتحقق من مطابقة ما كتبت، والحقيقة من الخادم في الخطوة 0 |
| اختبار خارج الميزة فشل | `routes_smoke_test` بعد تبديل مصدر الإشعارات | شغّل `flutter test` كاملاً قبل كل commit |
| كود معلّق بدل الحذف | البيانات التجريبية للإشعارات | احذف: `git show <commit>:<file>` يعيد أي نسخة قديمة |
| Model لا يساوي Entity | `ContactLinksModel` في اختبار الدعم | قارن بالـ Model في اختبارات البيانات |
| الوقت نص في domain | `time: 'منذ ساعتين'` | `DateTime` في domain، والعرض في presentation |
| اسم الخادم ≠ اسم التطبيق | `body` ↔ `text` | الـ Model يترجم، والـ Entity بلغة التطبيق |
| إصدار حالة بعد الإغلاق | `SearchCubit` القديم | `await` بعد كل `if (isClosed) return;` |
| شاشة تتجمد على التحميل | `guard` القديم لا يلتقط إلا 4 استثناءات | آخر فرع في guard: `on Exception` |
| أيقونة بالاتجاه المعاكس | مكبّر الصوت في الإشعارات | قارن بالتصميم بصرياً قبل التسليم |

الجزء الخامس

# رحلتنا في هذه الدورة، وما بعدها

ما بنيناه معاً، والمستوى الذي وصلت إليه، والطريق إلى تسليم المشروع.

## ما بنيناه في هذه الجلسة

1. **مراجعة المشروع (في بداية الجلسة):** كان analyze نظيفاً و142 اختباراً ناجحاً، لكن أشياء كثيرة **كانت** مكتوبة بأكثر من طريقة: ملفات الـ UseCases، والـ Models، ومكان الحالة، والتحقق المشترك.
2. **توحيد الاتفاقيات (تمّ):** وحّدنا كل ما سبق في commit واحد: ملف لكل UseCase بلا لاحقة، وملف لكل Model، والحالة في part، و `Validators` في core، و `guard` يلتقط كل `Exception`، وكل الألوان في `AppColors`. لم يبقَ منه شيء مختلف في الكود.
3. **كتابة المنهج:** `docs/FEATURE_WORKFLOW.md` و `CLAUDE.md`، ثم دمج الاشتراكات وإعادة الهيكلة مع main.
4. **أول ميزة بالمنهج، الإشعارات:** الخطوات 0 إلى 5 كاملة، من قراءة رد الخادم إلى صفحة مطابقة لفيجما، في 5 commits.
5. **هوية العمل:** كل الـ commits باسمك، بلا سطر Claude، مع إعادة كتابة التاريخ ورفعه بأمان.
6. **هذا الدليل**، وقالب CRUD مُختبَر يغطي كل حالات الاستخدام.

## أين وصلت

أنت الآن تستطيع، بلا مساعدة في المنطق:

- [ ] قراءة طلب في الخادم وتوثيق شكله وحدوده قبل كتابة سطر
- [ ] بناء الطبقات الثلاث لميزة، واحترام اتجاه الاعتماد
- [ ] تصميم حالة Cubit بحالات جلب وأفعال منفصلة، وكتابة اختباراتها
- [ ] ربط الميزة بـ DI والموجّه، وبناء صفحة بحالاتها الأربع من تصميم فيجما
- [ ] العمل بـ Git كما في فريق: فرع، commits صغيرة بأسماء واضحة، دمج، رفع

ما يبقى ليصبح عادة: التكرار. ميزتان أو ثلاث أخرى بالخطوات نفسها، وستكتبها دون الرجوع إلى هذا الدليل إلا للقوالب.

## الطريق إلى تسليم المشروع

بقية طلبات الخادم مرتبة من الأسهل إلى الأصعب. كل ميزة تمرّنك على حالة استخدام. ابدأ كل واحدة بالخطوة 0، فشكل هذه الردود لم يُتحقق منه بعد.

| # | الميزة | طلبات الخادم | ما تتمرن عليه |
|---|---|---|---|
| 1 | المعرض | `galleries/all` ، `galleries/paginate` | قراءة + صفحات |
| 2 | الرئيسية | `home-page/index` ، `sliders/all` ، `news/all` | عدة طلبات بالتوازي في Cubit واحد (كما في `HomeCubit` ) |
| 3 | الإنجازات | `profile/points` ، `profile/medals` ، `profile/cups` ، `users/top` | قراءة بتبويبات |
| 4 | البحث | `home-page/search` | بحث مع debounce |
| 5 | المواد والدورات والدروس | `subjects/*` ، `courses/*` ، `lessions/*` | تفاصيل متداخلة، صفحات، وفعل (حضور الدرس) |
| 6 | التعليقات | `comments/get-by-lession` ، `comments/add-comment` | CRUD: قراءة + إضافة |
| 7 | الاختبارات | `exams/my` ، `exams/attend-exam` ، `exams/result` ، `daily-activities/my` | إرسال إجابات وعرض نتيجة |
| 8 | اشتراكاتي | `subscriptions/my` ، `subscriptions/check-out-code` | قراءة + تحقق من كود |
| 9 | الحساب | `auth/resend` ، `auth/delete` | فعل صغير، وحذف مع تأكيد |

## قبل التسليم

> **راجع متطلبات «تمكين» الرسمية أولاً**
>
> هذه قائمة عامة لمشروع Flutter يُسلَّم للتقييم، لا بديل عن شروط الدورة.

- [ ] `flutter analyze` نظيف و `flutter test` ناجح على `main`
- [ ] README يشرح التشغيل والمعمارية، ويشير إلى المنهج
- [ ] كل شاشة تعمل بحالاتها الأربع على الخادم الحقيقي
- [ ] تاريخ Git واضح: commits صغيرة بأسماء تصف التغيير
- [ ] صور أو فيديو قصير لأهم المسارات (الدخول، الاشتراك، الإشعارات...)

### كيف تشرح مشروعك في دقيقتين

اختر زراً واحداً، «إضافة تعليق» مثلاً، وتتبّع رحلته بصوت عالٍ: الصفحة تستدعي الـ Cubit، الـ Cubit يصدر «جارٍ الإرسال» ويستدعي UseCase، المستودع يتحقق ثم يستدعي المصدر داخل `guard` ، المصدر يرسل عبر `ApiClient` ، والرد يعود Model ثم `Either` ثم حالة ثم واجهة. ثم قل لماذا: كل طبقة تُختبر وحدها، وتغيير الخادم يلمس ملفاً واحداً، والخطأ يصل المستخدم دائماً برسالة عربية. من يشرح هذا يفهم المعمارية، لا يحفظها.

# مسرد المصطلحات

| المصطلح | معناه هنا |
|---|---|
| Entity | كائن بيانات بلغة التطبيق، في domain |
| Model | Entity يعرف قراءة JSON، في data |
| Repository | عقد الميزة (domain) وتنفيذه (data) |
| DataSource | من يتحدث مع الخادم أو التخزين المحلي |
| UseCase | فعل واحد يطلبه المستخدم |
| Cubit / State | منطق الشاشة وحالتها، والواجهة ترسم الحالة |
| Either / Failure | نتيجة إما فشل برسالة أو قيمة |
| DI / get_it | مكان واحد ينشئ الكائنات ويربطها |
| Envelope | غلاف الرد `{message, data}` |
| Debounce | انتظار توقف الكتابة قبل الطلب |
| Pagination | جلب القائمة صفحةً صفحة |
| Smoke test | اختبار سريع أن كل مسار يُفتح بلا خطأ |
| Fast-forward | دمج بلا commit دمج، حين لم يتحرك main |
| `--force-with-lease` | رفع بالقوة يرفض إن تغيّر الفرع على GitHub منذ آخر جلب |